

Software Engineering
Lab Assignment 7
Public Bus Ticket Booking System

Requirement Analysis & Design

Student ID: 202401138
Name: Dev Parmar

Parts A, B, C and D

Contents

1	Part A – Class Identification	3
1.1	System Overview	3
1.2	Bus	3
1.3	Trip	4
1.4	Customer	5
1.5	Booking	5
1.6	Payment	6
1.7	Enumerations	6
1.7.1	BusType	6
1.7.2	BookingStatus	6
1.7.3	PaymentStatus	6
2	Part B – Relationships and Multiplicity	7
2.1	UML Class Diagram	7
2.2	Bus – Trip	7
2.3	Customer – Booking	8
2.4	Trip – Booking	8
2.5	Booking – Payment	8
2.6	Relationship Summary	8
3	Part C – Design Decisions	9
3.1	Overall Booking Workflow	9
3.2	Trip Search	10
3.3	Via Information	11
3.4	Seat Availability	11
3.5	Independent Seat Availability for Trips	11
3.6	Seat Validation	12
3.7	Price Calculation	12
3.8	Payment Processing	12
3.9	Cancellation and Refund	13
3.10	Booking as Ticket	13
3.11	Minimum Viable Design	13
4	Part D – Test Cases	15
4.1	TC01 – Successful Booking	15
4.2	TC02 – Invalid or Duplicate Seat	15
4.3	TC03 – Payment Failure	16
4.4	TC04 – Cancellation More Than 24 Hours Before Departure	16
4.5	TC05 – Cancellation Between 12 and 24 Hours Before Departure	16
4.6	TC06 – Cancellation Less Than 12 Hours Before Departure	17

4.7	TC07 – Multiple Bookings	17
4.8	Test Summary	17
4.9	Overall Test Result	17

1. Part A – Class Identification

1.1 System Overview

The system is a public bus ticket booking system in which a customer searches for available bus trips based on a source and destination.

The customer first enters the source and destination. The system then finds matching trips, filters the available options, and sorts them by departure time. The customer selects a trip and views its available seats and price. After selecting one or more seats, the customer proceeds to payment. A successful payment confirms the booking, while a failed payment does not confirm the booking.

A confirmed booking contains the information required to represent the customer's ticket.

The minimum viable design contains five classes:

1. Bus
2. Trip
3. Customer
4. Booking
5. Payment

Three enumerations are also used:

- BusType
- BookingStatus
- PaymentStatus

1.2 Bus

Attributes:

- busNumber : String
- busType : BusType
- seatCount : int

Responsibilities:

- Represents a bus used for scheduled trips.
- Stores the bus number and bus type.

- Defines the total number of fixed numbered seats.
- Provides the valid seat range for the bus.

No separate **Seat** class is required. Individual seats are represented using integer seat numbers.

1.3 Trip

Attributes:

- tripId : String
- source : String
- destination : String
- via : String
- departureDateTime : LocalDateTime
- bus : Bus
- pricePerSeat : double
- bookedSeats : Set<Integer>

Responsibilities:

- Represents one scheduled bus trip.
- Stores the source, destination and intermediate path information.
- Stores the departure date and time.
- Identifies the bus assigned to the trip.
- Stores the price per seat.
- Maintains the seats booked for that particular trip.
- Provides available-seat information.
- Supports reserving and releasing seats.

The **via** attribute represents intermediate locations or the path followed by the trip. No separate **Route** class is used.

1.4 Customer

Attributes:

- `customerId` : `String`
- `name` : `String`
- `phone` : `String`

Responsibilities:

- Represents the customer using the system.
- Initiates the search for trips.
- Selects a trip and seats.
- Creates and manages bookings.

1.5 Booking

Attributes:

- `bookingId` : `String`
- `customer` : `Customer`
- `trip` : `Trip`
- `selectedSeats` : `Set<Integer>`
- `totalAmount` : `double`
- `status` : `BookingStatus`
- `payment` : `Payment`

Responsibilities:

- Represents the customer's booking transaction.
- Stores the selected trip and seats.
- Calculates the total booking amount.
- Confirms the booking after successful payment.
- Handles cancellation.
- Calculates the applicable refund.

- Releases seats when a booking is cancelled.
- Represents the confirmed ticket/booking information.

A separate `Ticket` class is not required. A confirmed `Booking` contains the information needed to represent the customer's ticket.

1.6 Payment

Attributes:

- `paymentId` : `String`
- `amount` : `double`
- `status` : `PaymentStatus`

Responsibilities:

- Simulates the payment transaction.
- Stores the payment amount.
- Records whether the payment succeeded or failed.

1.7 Enumerations

1.7.1 `BusType`

`AC_SLEEPER`
`AC_SEATER`
`NON_AC`

1.7.2 `BookingStatus`

`PENDING`
`CONFIRMED`
`CANCELLED`

1.7.3 `PaymentStatus`

`SUCCESS`
`FAILED`

2. Part B – Relationships and Multiplicity

The system uses four main class relationships.

2.1 UML Class Diagram

The UML class diagram below represents the final class structure, attributes, methods, enumerations, and relationships of the Public Bus Ticket Booking System.

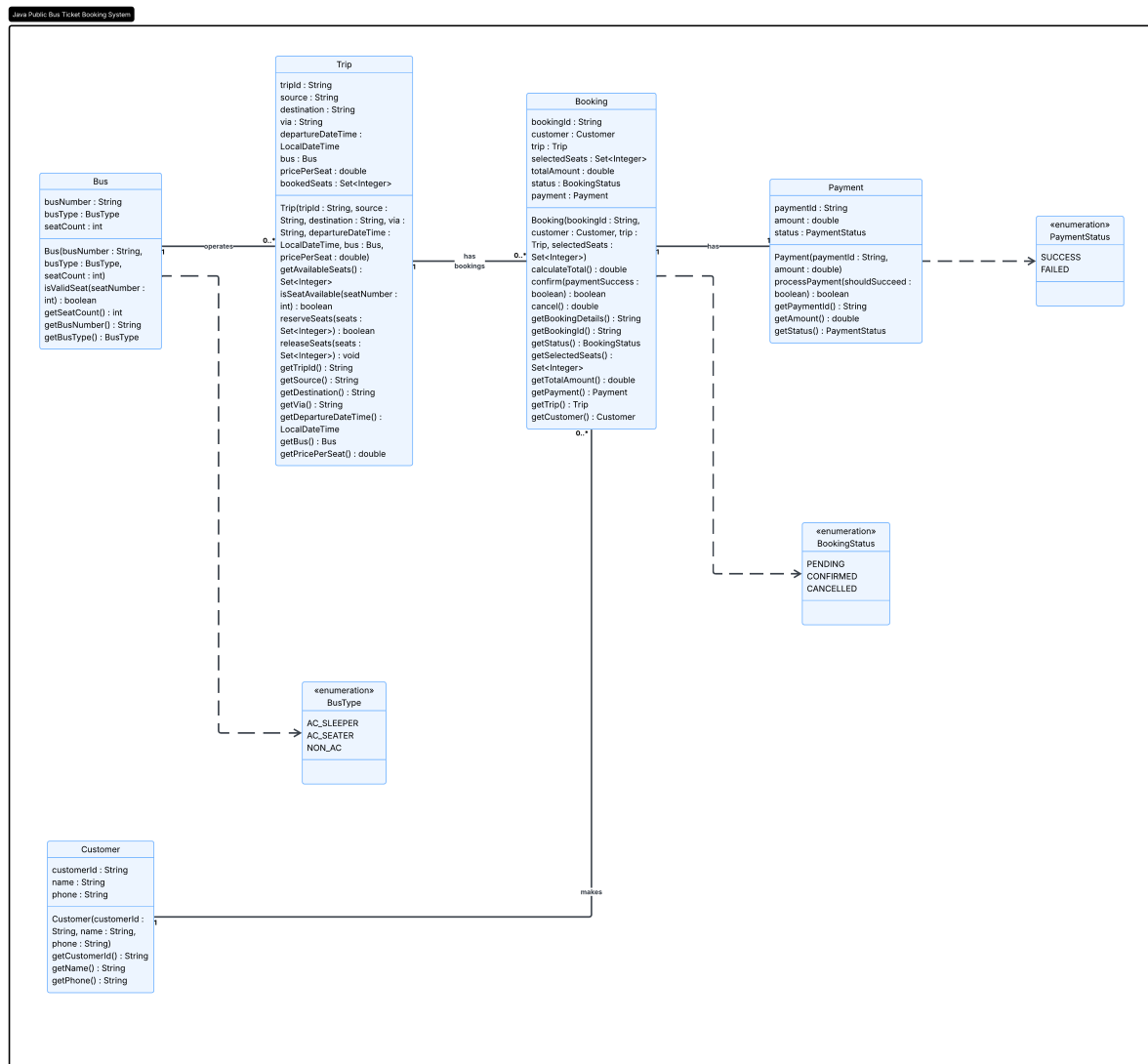


Figure 1: UML Class Diagram of the Public Bus Ticket Booking System

2.2 Bus – Trip

Bus 1 ----- 0..* Trip

A bus can be assigned to multiple trips over time. Each trip uses exactly one bus.

The same bus can therefore operate different trips while maintaining separate seat availability for each trip.

2.3 Customer – Booking

Customer 1 ----- 0..* Booking

A customer can have zero or more bookings. Each booking belongs to exactly one customer.

2.4 Trip – Booking

Trip 1 ----- 0..* Booking

A trip can have zero or more bookings. Each booking is associated with exactly one trip.

2.5 Booking – Payment

Booking 1 ----- 1 Payment

Each booking has one simulated payment. The payment records whether the transaction succeeds or fails.

2.6 Relationship Summary

Relationship	Multiplicity
Bus – Trip	1 : 0..*
Customer – Booking	1 : 0..*
Trip – Booking	1 : 0..*
Booking – Payment	1 : 1

Table 1: Class Relationships and Multiplicity

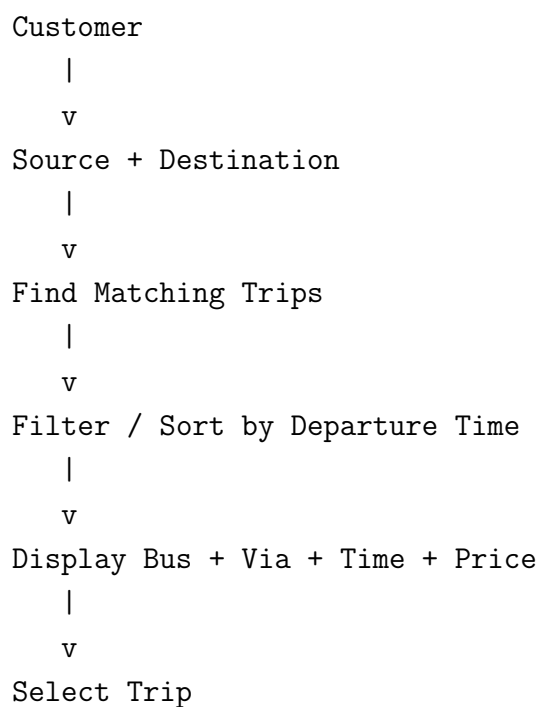
3. Part C – Design Decisions

3.1 Overall Booking Workflow

The system follows the customer's booking workflow:

1. The customer enters the source and destination.
2. The system finds trips matching the selected source and destination.
3. The available trips are filtered and sorted by departure time.
4. The system displays the bus, via information, departure time and price.
5. The customer selects a trip.
6. The system displays the available seats for the selected trip.
7. The customer selects one or more seats.
8. The system calculates the total booking amount.
9. The customer proceeds to simulated payment.
10. If payment succeeds, the booking is confirmed.
11. The confirmed booking acts as the customer's ticket.
12. A confirmed booking can later be cancelled according to the cancellation policy.

The workflow is:



```

|
v
Available Seats
|
v
Select Seat(s)
|
v
Calculate Total Amount
|
v
Payment
|
+-----+
|               |
SUCCESS         FAILED
|               |
v               v
CONFIRMED       REJECTED
|
v
Booking / Ticket Confirmation

```

3.2 Trip Search

The customer searches using a source and destination.

For example:

```

Source      : Ahmedabad
Destination : Mumbai

```

The system finds trips for which:

```

trip.source == source
AND
trip.destination == destination

```

The matching trips are then sorted according to `departureDateTime`.

For example:

08:00	Bus 101	Via: Vadodara, Surat	Rs.800
14:00	Bus 105	Via: Vadodara	Rs.750
22:00	Bus 110	Via: Bharuch, Surat	Rs.900

The customer selects one of these trips.

3.3 Via Information

The `via` attribute stores intermediate locations or path information for a trip.

For example:

```
Source      : Ahmedabad
Via         : Vadodara, Surat
Destination : Mumbai
```

A separate class is not required for this information in the minimum viable implementation.

3.4 Seat Availability

The `Bus` class stores the total number of seats.

For example:

```
Bus 101
seatCount = 40
```

Therefore, the valid seat numbers are:

1, 2, 3, ..., 40

The `Trip` class stores the seats already booked for that specific trip.

For example:

```
Trip T01
bookedSeats = {4, 7, 12}
```

The available seats are all valid bus seats except those contained in `bookedSeats`.

3.5 Independent Seat Availability for Trips

Seat availability is maintained by `Trip`, rather than by `Bus`.

For example:

```
Bus 101
Seats: 1 - 40
```

```
Trip T01
bookedSeats = {14}
```

```
Trip T02
bookedSeats = {}
```

Therefore:

Seat 14 on T01 -> BOOKED

Seat 14 on T02 -> AVAILABLE

This ensures that booking a seat on one trip does not make the same seat unavailable on another trip operated by the same bus.

3.6 Seat Validation

Before a seat is selected, the system checks whether the seat number is within the bus's valid range:

```
1 <= seatNumber <= bus.seatCount
```

It also checks whether the seat is already booked for the selected trip:

```
seatNumber not in trip.bookedSeats
```

An invalid or already-booked seat is therefore rejected.

3.7 Price Calculation

The fare is stored in the selected trip using `pricePerSeat`.

The total booking amount is:

$$TotalAmount = NumberOfSelectedSeats \times PricePerSeat$$

For example, if two seats cost Rs.800 each:

$$TotalAmount = 2 \times 800 = Rs.1600$$

The price is associated with the trip because different trips can have different fares.

3.8 Payment Processing

Payment is simulated rather than connected to a real payment gateway.

For a successful payment:

```
Payment.status = SUCCESS
```

```
Booking.status = CONFIRMED
```

For a failed payment:

```
Payment.status = FAILED
```

```
Booking is not confirmed
```

```
Selected seats are released
```

This ensures that failed payments do not result in permanently reserved seats.

3.9 Cancellation and Refund

A confirmed booking can be cancelled. The refund depends on the time remaining before departure.

The implementation uses the following policy:

Time Before Departure	Refund
More than 24 hours	100%
12–24 hours	50%
Less than 12 hours	0%

Table 2: Cancellation and Refund Policy

After cancellation:

```
Booking.status = CANCELLED
```

The selected seats are also released from the trip's `bookedSeats` set.

3.10 Booking as Ticket

A separate `Ticket` class is not used.

The confirmed `Booking` already contains the information required to represent the customer's ticket:

```
Booking
  bookingId
  customer
  trip
  selectedSeats
  totalAmount
  status
```

After successful payment, the confirmed booking is displayed as the ticket/booking confirmation.

3.11 Minimum Viable Design

The final design intentionally contains only the classes necessary for the minimum viable implementation:

```
Bus
Trip
Customer
Booking
Payment
```

The following concepts are represented directly rather than as separate classes:

- Source, destination and intermediate path are attributes of **Trip**.
- Individual seats are represented by integer seat numbers.
- Seat availability is represented by **Trip.bookedSeats**.
- Ticket information is represented by a confirmed **Booking**.
- Refund calculation is handled during booking cancellation.

This design keeps the implementation simple while supporting the required trip search, seat selection, pricing, payment, booking, cancellation and refund functionality.

4. Part D – Test Cases

The implemented system was tested using the test cases defined for booking, seat validation, payment processing, cancellation, refund calculation and multiple bookings.

All seven test cases were executed successfully.

4.1 TC01 – Successful Booking

Objective: Verify that a customer can successfully book available seats and complete a successful payment.

Input / Condition:

- Select a valid source and destination.
- Select an available trip.
- Select available seats.
- Enter valid passenger details.
- Select a payment method.
- Choose successful payment.

Expected Result:

The payment should succeed, the booking should be confirmed, the selected seats should be reserved, and the booking should appear in the My Bookings section.

Actual Result:

The booking was successfully created and confirmed. The selected seats were reserved and the booking appeared in My Bookings.

Status: PASS

4.2 TC02 – Invalid or Duplicate Seat

Objective: Verify that invalid or already-booked seats cannot be selected.

Input / Condition:

An invalid seat number or a seat that has already been booked is selected.

Expected Result:

The system should reject the seat selection and prevent an invalid booking.

Actual Result:

Invalid and already-booked seats were rejected by the system. The booking could not proceed with an unavailable seat.

Status: PASS

4.3 TC03 – Payment Failure

Objective: Verify that a failed payment does not confirm the booking and that the selected seats are released.

Input / Condition:

A valid trip and available seats are selected, followed by simulated payment failure.

Expected Result:

The payment should be marked as failed, the booking should not be confirmed, and the selected seats should be released.

Actual Result:

The payment failure was handled correctly. The booking was not confirmed and the selected seats were released.

Status: PASS

4.4 TC04 – Cancellation More Than 24 Hours Before Departure

Objective: Verify the cancellation and full-refund behavior when the booking is cancelled more than 24 hours before departure.

Input / Condition:

A confirmed booking is cancelled more than 24 hours before the scheduled departure.

Expected Result:

The booking should be cancelled and 100% of the booking amount should be calculated as the refund.

Actual Result:

The booking was successfully cancelled and the applicable refund amount was displayed according to the cancellation policy.

Status: PASS

4.5 TC05 – Cancellation Between 12 and 24 Hours Before Departure

Objective: Verify the partial-refund behavior when the booking is cancelled between 12 and 24 hours before departure.

Input / Condition:

A confirmed booking is cancelled between 12 and 24 hours before the scheduled departure.

Expected Result:

The booking should be cancelled and 50% of the booking amount should be calculated as the refund.

Actual Result:

The booking was successfully cancelled and the applicable 50% refund was calculated and displayed.

Status: PASS

4.6 TC06 – Cancellation Less Than 12 Hours Before Departure

Objective: Verify the no-refund behavior when the booking is cancelled less than 12 hours before departure.

Input / Condition:

A confirmed booking is cancelled less than 12 hours before the scheduled departure.

Expected Result:

The booking should be cancelled and the refund amount should be Rs.0.

Actual Result:

The booking was successfully cancelled and no refund was calculated for the booking.

Status: PASS

4.7 TC07 – Multiple Bookings

Objective: Verify that multiple bookings can be created and managed independently.

Input / Condition:

Two or more valid bookings are created for available trips and seats.

Expected Result:

All valid bookings should be stored independently. Cancelling one booking should not affect the other booking.

Actual Result:

Multiple bookings were successfully created and stored. Each booking was maintained independently.

Status: PASS

4.8 Test Summary

The following table summarizes the execution of all seven test cases.

Test Case	Scenario	Status
TC01	Successful Booking	PASS
TC02	Invalid or Duplicate Seat	PASS
TC03	Payment Failure	PASS
TC04	Cancellation > 24 Hours	PASS
TC05	Cancellation 12–24 Hours	PASS
TC06	Cancellation < 12 Hours	PASS
TC07	Multiple Bookings	PASS

Table 3: Test Case Execution Summary

4.9 Overall Test Result

All seven defined test cases were executed successfully.

Total Test Cases : 7
Passed : 7
Failed : 0
Overall Status : PASS

The testing confirms that the implemented system supports the required minimum viable booking workflow, including trip selection, seat validation, payment handling, booking confirmation, cancellation, refund calculation and management of multiple bookings.